

2520. Count the Digits That Divide a Number

Input: num = 121

Output: 2

Explanation: 121 is divisible by 1, but not 2. Since 1 occurs twice as a digit, we return 2.

```
class Solution {
    public int countDigits(int num) {
        int cnt = 0;
        int div = 1;
        while (div <= num) {
            int ed = (num % (div * 10)) / div;
            if (num % ed == 0) {
                cnt++;
            }
            div *= 10;
        }
        return cnt;
    }
}
```

2652. Sum Multiples

Input: n = 7

Output: 21

Explanation: Numbers in the range [1, 7] that are divisible by 3, 5, or 7 are 3, 5, 6, 7. The sum of these numbers is 21.

```
class Solution {
    public int sumOfMultiples(int n) {
        int sum = 0;
        for(int i=1;i<=n;i++){
            if(i%3==0 || i%5==0 || i%7==0){
                sum += i;
            }
        }
        return sum;
    }
}
```

2160. Minimum Sum of Four Digit Number After Splitting Digits

You are given a **positive** integer num consisting of exactly four digits. Split num into two new integers new1 and new2 by using the **digits** found in num. **Leading zeros** are allowed in new1 and new2, and **all** the digits found in num must be used.

- For example, given num = 2932, you have the following digits: two 2's, one 9 and one 3. Some of the possible pairs [new1, new2] are [22, 93], [23, 92], [223, 9] and [2, 329].

Input: num = 2932

Output: 52

Explanation: Some possible pairs [new1, new2] are [29, 23], [223, 9], etc. The minimum sum can be obtained by the pair [29, 23]: 29 + 23 = 52.

```

class Solution {
    public int minimumSum(int num) {
        int min1 = 9, min2 = 9, max1 = 0, max2 = 0;
        int div = 1;
        while(div<=num){
            int ed = (num%(div*10))/div;
            if(ed<=min1){
                min2 = min1;
                min1 = ed;
            }
            else if(ed<=min2){
                min2 = ed;
            }
            div = div * 10;
        }
        div = 1;
        while(div<=num){
            int ed = (num%(div*10))/div;
            if(ed>=max1){
                max2 = max1;
                max1 = ed;
            }
            else if(ed>=max2){
                max2 = ed;
            }
            div = div * 10;
        }
        return (min1*10+max1) + (min2*10+max2);
    }
}

```

num = 2932

split → [1, 3], [3, 1] ✗

split → [2, 2] ✓

① → 29 + 32 = 61 ✗

② → 29 + 23 = 52 ✓

min1=9, min2=9

max1=0, max2=0.

(min1*10+max1) + (min2*10+max2)

Strong Number

ip: 145

= 1! + 4! + 5!

= 1 + 24 + 120

= 145

Op:- True

Note:-

2 functions needed.

1 → factorial (int)

2 → Strong Number (boolean)

```

public static int Fact(int n){
    int total = 1;
    for(int i=1;i<=n;i++){
        total *= i;
    }
    return total;
}
public static boolean strongNumber(int n){
    int sum = 0;
    int div = 1;
    while(div<=n){
        int ed = (n%(div*10))/div;//4
        sum += Fact(ed);
        div = div*10;
    }
    return sum==n;
}

```

9. Palindrome Number

Input: x = 121

Output: true

Explanation: 121 reads as 121 from left to right and from right to left.

```
public boolean isPalindrome(int n) {  
    int rev = 0;  
    if(n<0) return false;  
    int temp = n;  
    while(n>0){  
        int ed = n%10;;  
        rev = rev*10 + ed;  
        n/=10;  
    }  
    return rev == temp;  
}
```

3099. Harshad Number

An integer divisible by the **sum** of its digits is said to be a **Harshad** number. You are given an integer `x`. Return *the sum of the digits* of `x` if `x` is a **Harshad** number, otherwise, return `-1`.

Input: x = 18

Output: 9

Explanation:

The sum of digits of `x` is 9. 18 is divisible by 9. So 18 is a Harshad number and the answer is 9.

```
class Solution {  
    public int sumOfTheDigitsOfHarshadNumber(int n) {  
        int sum = 0;  
        int div = 1;  
        while(div<=n){  
            int ed = (n%(div*10))/div;  
            sum += ed;  
            div = div * 10;  
        }  
        if(n%sum==0) return sum;  
        return -1;  
    }  
}
```

1323. Maximum 69 Number

You are given a positive integer `num` consisting only of digits 6 and 9.

Return *the maximum number* you can get by changing **at most** one digit (6 becomes 9, and 9 becomes 6).

Input: num = 9669

Output: 9969

Explanation:

Changing the first digit results in 6669.

Changing the second digit results in 9969.

Changing the third digit results in 9699.

Changing the fourth digit results in 9666.

The maximum number is 9969.

6-9 9-6 x
96 → 99.
9669 → 9699, 9969 ✓
96 $\xrightarrow{+3}$ 99
969 $\xrightarrow{+30}$ 999

```

public int maximum69Number (int n) {
    int val = 3;
    int num = n;
    int ans = n;
    while(n!=0){
        int ld = n%10;
        if(ld==6){//96
            ans = num + val;
        }

        val *= 10;
        n = n/10;
    }
    return ans;
}

```

509. Fibonacci Number

The **Fibonacci numbers**, commonly denoted $F(n)$ form a sequence, called the **Fibonacci sequence**, such that each number is the sum of the two preceding ones, starting from 0 and 1. That is,

$$F(0) = 0, F(1) = 1$$

$$F(n) = F(n - 1) + F(n - 2), \text{ for } n > 1.$$

Input: $n = 2$

Output: 1

Explanation: $F(2) = F(1) + F(0) = 1 + 0 = 1.$

0	1	2	3	4	5	6	7	8
0	1	1	2	3	5	8	13	21 ...
a	b	add						
a	b							

Arrows indicate the calculation of 'add' from 'a' and 'b' at each step.

```

class Solution {
    public int fib(int n) {
        int a = 0, b = 1, add = 0;
        if (n == 0 || n == 1)
            return n;
        for (int i = 2; i <= n; i++) {
            add = a + b;
            a = b;
            b = add;
        }
        return add;
    }
}

```

1) Declare $a = 0$, $b = 1$, $add = 0$.

2) If $(n == 0 || n == 1) \rightarrow \text{return } n;$

```

3) for (i = 2; i <= n; i++) {
    add = a + b;
    a = b;
    b = add;
}
return add;

```

1137. N-th Tribonacci Number

The Tribonacci sequence T_n is defined as follows:

$T_0 = 0$, $T_1 = 1$, $T_2 = 1$, and $T_{n+3} = T_n + T_{n+1} + T_{n+2}$ for $n \geq 0$

Given n , return the value of T_n .

Input: $n = 4$

Output: 4

Explanation:

$T_3 = 0 + 1 + 1 = 2$

$T_4 = 1 + 1 + 2 = 4$

```
public int tribonacci(int n) {
    int a = 0, b = 1, c = 1, add=0;
    if(n==0) return 0;
    if(n==1 || n==2) return 1;
    for(int i=3;i<=n;i++){
        add = a+b+c;
        a = b;
        b = c;
        c = add;
    }
    return add;
}
```

0 (1 1 2) return 1
 $a=0$ $b=1$ $c=1$ $add=0$.

```
for (i=3; i<=n; i++){
    add = a+b+c;
    a = b;
    b = c;
    c = add;
}
```

0 1 1 2 4 7 13
 a b c add
 a b c

202. Happy Number

Input: $n = 19$

Output: true

Explanation:

$1^2 + 9^2 = 82$

$8^2 + 2^2 = 68$

$6^2 + 8^2 = 100$

$1^2 + 0^2 + 0^2 = 1$

$n = 19$

$= 9^2 + 1^2 = 81 + 1 = 82 \rightarrow n$

$= 8^2 + 2^2 = 64 + 4 = 68 \rightarrow n$

$= 6^2 + 8^2 = 36 + 64 = 100 \rightarrow n$

$= 1^2 + 0^2 + 0^2 = 1$

```
class Solution {
    public boolean isHappy(int n) {
        int sum = n;
        while(sum>=10){
            sum = 0;
            while(n!=0){
                int ld = n%10;
                sum += ld*ld;
                n/=10;
            }
            n = sum;
        }
        return n==1 || n==7;
    }
}
```

7 is also a happy number.

$7^2 = 49$

$4^2 + 9^2 = 16 + 81 = 97$

$9^2 + 7^2 = 81 + 49 = 130$

$1^2 + 3^2 + 0^2 = 1 + 9 + 0 = 10$

$1^2 + 0^2 = 1 + 0 = 1$

70. Climbing Stairs

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Input: $n = 3$

Output: 3

Explanation: There are three ways to climb to the top.

1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step

$n = 2$

1) 1, 1

2) 2

$n = 3$

1) 1, 1, 1

2) 2, 1

3) 1, 2

$n = 4$

1) 1, 1, 1, 1

2) 2, 2

3) 1, 1, 2

4) 1, 2, 1

5) 2, 1, 1

$n = 5$

1) 1, 1, 1, 1, 1

2) 2, 2, 1

3) 2, 1, 2

4) 1, 2, 2

5) 1, 2, 1, 1

6) 2, 1, 1, 1

7) 1, 1, 2, 1

8) 1, 1, 1, 2

```
public int climbStairs(int n) {  
    int a = 1, b = 2, add = 0;  
    if(n==1 || n==2){  
        return n;  
    }  
    for(int i=3; i<=n; i++){  
        add = a+b;  
        a = b;  
        b = add;  
    }  
    return add;  
}
```

1, 2, 3, 5, 8

It starts from 1.

231. Power of Two

Input: $n = 16$

Output: true

Explanation: $2^4 = 16$

```
class Solution {  
    public boolean isPowerOfTwo(int n) {  
        if(n==0) return false;  
        if(n==1) return true;  
        while(n%2==0){  
            n/=2;  
        }  
        return n==1;  
    }  
}
```

326. Power of Three

Input: $n = 27$

Output: true

Explanation: $27 = 3^3$

```
class Solution {
    public boolean isPowerOfThree(int n) {
        if(n==0) return false;
        if(n==1) return true;
        while(n%3==0){
            n/=3;
        }
        return n==1;
    }
}
```

263. Ugly Number

Input: $n = 6$

Output: true

Explanation: $6 = 2 \times 3$

```
public boolean isUgly(int n) {
    if(n<=0) return false;
    while(n%2==0){
        n/=2;
    }
    while(n%3==0){
        n/=3;
    }
    while(n%5==0){
        n/=5;
    }
    return n==1;
}
```

Count factors

ip: $28 \rightarrow 1, 2, 4, 7, 14, \text{---} 28.$

op: 6

ip: $100 \rightarrow 1, 2, 4, 5, 10, 20, 25, 50, \text{---} 100$

op: 9.

```
public static int Countfactor1(int n){
    int cnt = 0;
    for(int i=1; i<=n; i++){
        if(n%i==0){
            cnt++;
        }
    }
    return cnt;
}
```

Any compiler.

10^8 iteration = 1 sec.

1 iteration = $\frac{1}{10^8}$ sec.

10^9 iteration = $\frac{1}{10^8}$ sec.

10 sec.

10^{10} iteration = $\frac{1}{10^8}$ sec.

10^{10} sec ≈ 316 yn

```

public static int Countfactor2(int n){
    int cnt = 1;
    for(int i=1; i<=n/2; i++){
        if(n%i==0){
            cnt++;
        }
    }
    return cnt;
}

```

$\approx 158 \text{ yr.}$

power of
optimization.

i	$n/i = 28$
1	$\leq 28/1 = 28 \rightarrow 2 \text{ cnt}$
2	$\leq 28/2 = 14 \rightarrow 2 \text{ cnt}$
4	$\leq 28/4 = 7 \rightarrow 2 \text{ cnt}$
7	$> 28/7 = 4$
$i \leq n/i$	

i	n/i
1	$\leq 100/1 = 100 \quad 2 \text{ cnt}$
2	$\leq 100/2 = 50 \quad 2 \text{ cnt}$
4	$\leq 100/4 = 25 \quad 2 \text{ cnt}$
5	$\leq 100/5 = 20 \quad 2 \text{ cnt}$
10	$\leq 100/10 = 10 \quad 1 \text{ cnt}$
20	$100/20 = 5$

```

public static int Countfactor3(int n){
    int cnt = 0;
    for(int i=1; i<=n/i; i++){
        if(n%i==0){
            if(i==n/i) cnt++;
            else cnt+=2;
        }
    }
    return cnt;
}

```

$i \leq n/i$
 $i * i \leq n$
 $i^2 \leq n$
 $i \leq \sqrt{n}$

$T_c \approx O(\sqrt{n})$

507. Perfect Number

Input: num = 28

Output: true

Explanation: $28 = 1 + 2 + 4 + 7 + 14$
1, 2, 4, 7, and 14 are all divisors of 28.

```
public boolean checkPerfectNumber(int n) {  
    int sum = 0;  
    for(int i=1;i<=n/i;i++){  
        if(n%i==0){  
            if(i==n/i){  
                sum += i;  
            }else{  
                sum += i+n/i;  
            }  
        }  
    }  
    sum = sum-n;  
    return n==sum;  
}
```

H/w Prime Number.

ip: 13

op: True.